1. The City (Twin)

Written by the Twin team · folder `twin/` · contract `2026-09-17` (the Guardian team's dictionary) · "What is really happening?"

1.1 Role and goals

The Twin **is the city**. It simulates two city blocks in real time: cars, pedestrians, signals, three emergency units, 24 buildings and the rain on seven streets. It is the only layer that knows what really happens. The City Brain and the Guardian see it as a real control room would: through sensor readings the Twin signs, and through the actuators they call. The Twin never dispatches by itself; an ambulance leaves its bay only when a `dispatch_unit` call arrives.

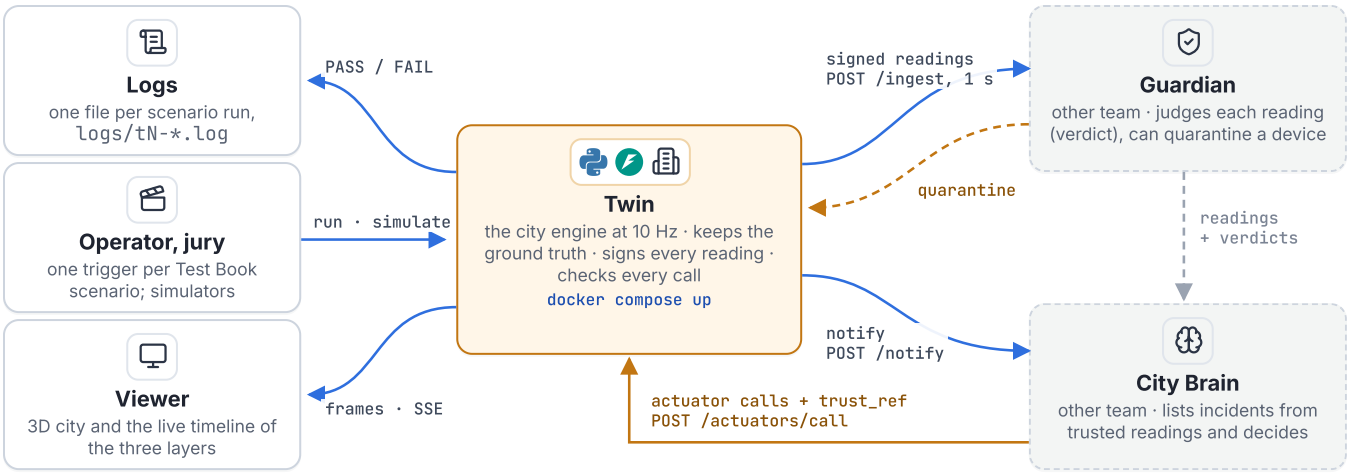


Figure 1.1. The Twin and its partners, with the protocol on each link (HTTP and JSON). Blue: what the Twin sends; amber: what other layers ask it to do. Dashed cards are the other teams' layers: mocks here, until `GUARDIAN_URL` or `BRAIN_URL` points at a real one.

Four rules shape the design:

1. **The truth stays inside.** The engine records every accident, fire and flood, but the other layers learn of it only from sensors, which can be late or wrong: a caller places a crash 3 m off, and one in ten names the wrong category.
2. **Nothing moves without a call.** Units, road barriers and pumps change only through actuators, and every call is checked: the layer's key, the contract version, the parameters, and that a Brain call names the Guardian verdict it relies on (`trust_ref`).
3. **Every reading can be checked.** Each one carries a sequence number per sensor and an HMAC-SHA256 over its canonical JSON, and honest sensors agree with each other (rain against water level, smoke against air quality).
4. **One trigger, one log.** A scenario starts with one button or command, is checked against the events expected from all three layers, and ends with `PASS` or `FAIL` in its log (R2).

Today the Twin runs alone with its two mocks from one command, and the four Test Book scenarios pass end to end through them. It has 303 Python and 94 Vitest tests; the traffic rules are checked from geometry, not engine internals (`tools/check_rules.py`).

1.2 Architecture

The Twin is one Python service around a seeded engine. Every 0.1 s the engine advances the whole city by one tick; the service turns what happened into readings and live frames, and incoming calls into changes in the engine. The API talks only to a `CitySource` interface, so another city or another transport (MCP) is a new adapter, not a rewrite.

From the Phase 0 design to what runs

Our Phase 0 report planned a Twin on SUMO, MCP servers and mutual TLS. We kept its ideas (subjects with traits, intent apart from physics, sequence numbers and HMACs, an LLM teacher distilled into a student) and changed the means, so its diagrams no longer describe the code and are redrawn.

Phase 0 plan	What runs now	Why we changed it
SUMO through TraCI for driving	our own engine: IDM, MOBIL, a kinematic bicycle model	junction, crosswalk and siren rules had to be ours; no second process to keep in step
External, Control and Logs MCP servers	HTTP and JSON, server-sent events for streams, behind a seam	our decision of 14 Oct, to keep the first link simple; MCP stays one adapter away (§1.4)
NDJSON log stream over mutual TLS	one signed reading per request, HMAC and sequence number per sensor	the Guardian's contract asks for exactly this envelope; TLS is still open (§1.6)
LLM teacher, XGBoost student, LLM fallback and canary checks	the teacher and the student; a random choice if the student cannot score	no network or API key on stage; a demo cannot depend on an LLM answering

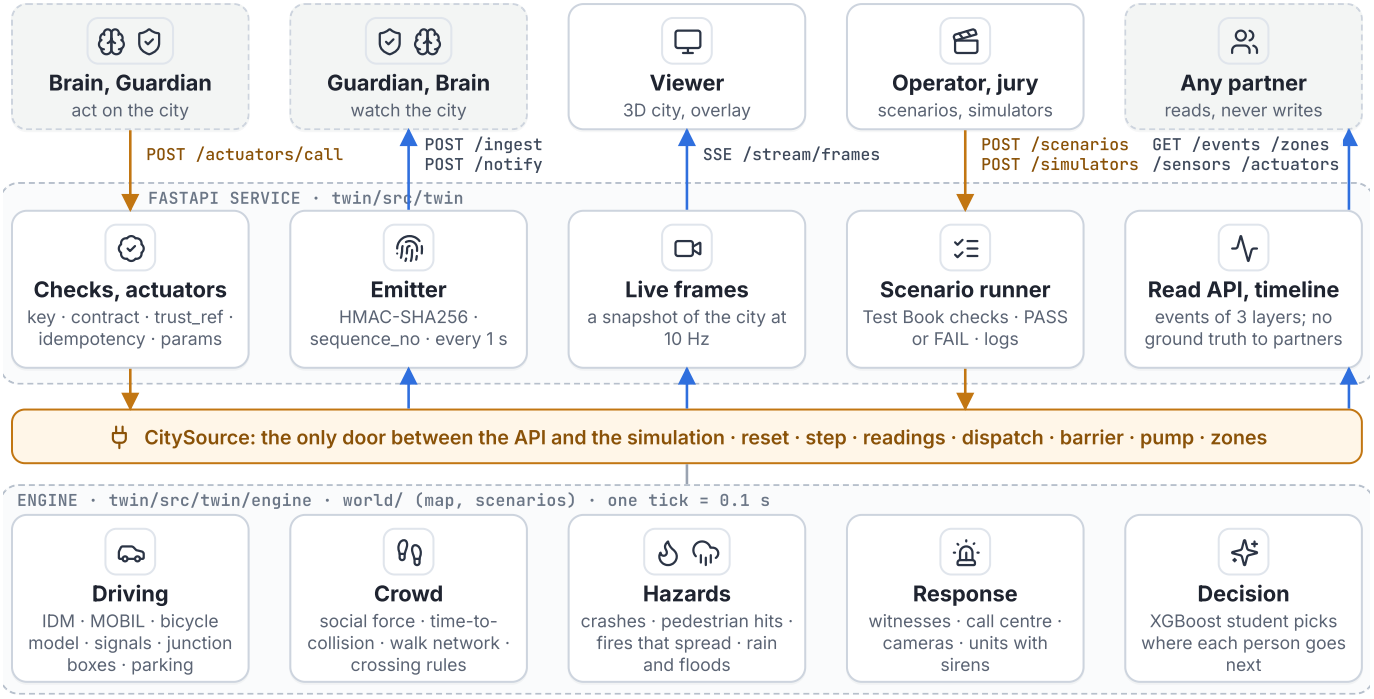


Figure 1.2. Inside the Twin. Five columns, five paths: acting (amber, down), sensing (blue, up), showing, testing, reading. Every path crosses the `CitySource` seam (amber), so the engine never sees HTTP and the API never sees engine objects.

1.2.1 City plan and zones

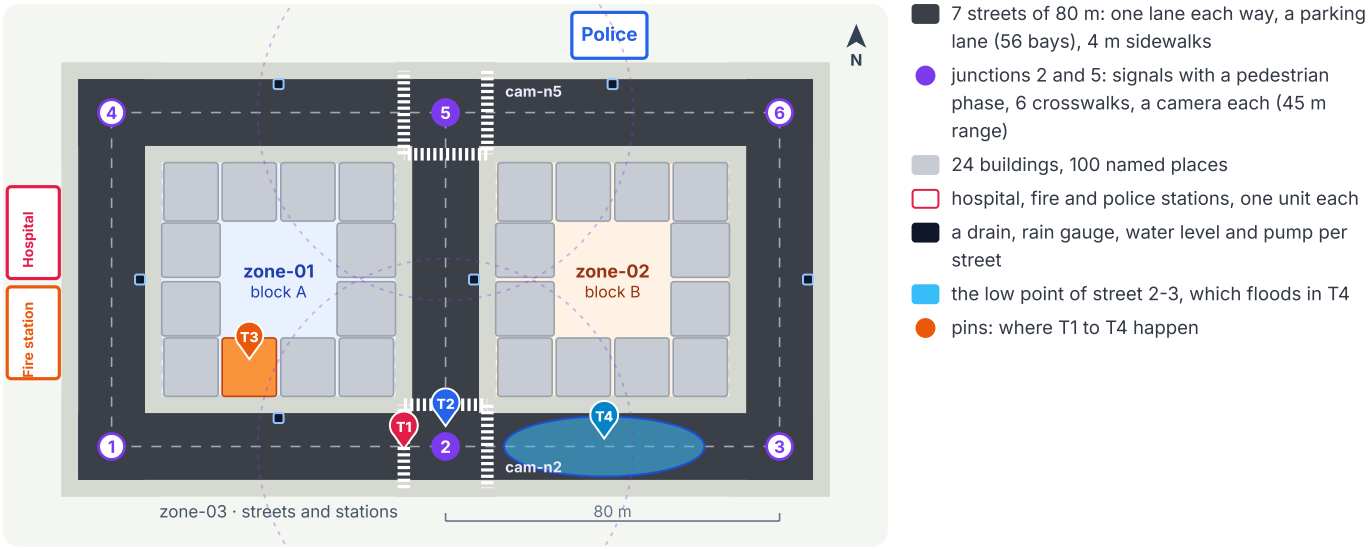


Figure 1.3. The two-block map, drawn to scale from the Twin's own map (`GET /live/map`). Three zones answer the contract's open item on granularity: `zone-01` and `zone-02` are the blocks, `zone-03` the streets and stations; `GET /zones` gives their bounds, adjacency and facilities, so no layer hard-codes geometry.

1.2.2 One tick of the city

The order inside a tick is fixed, and it decides who reacts to what. Signals and water are settled before anyone moves, accidents are found after everyone has moved, and observers look last.

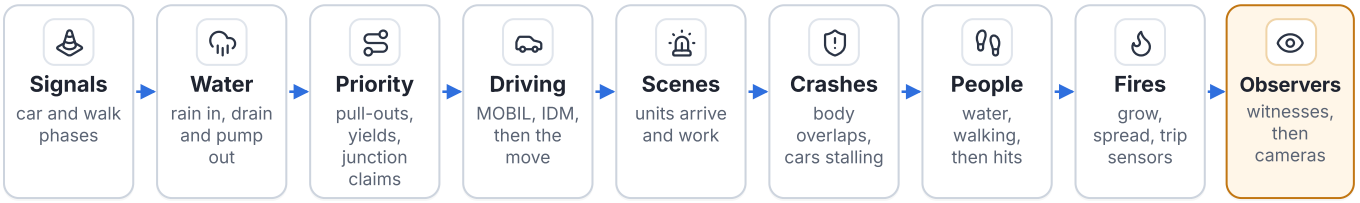


Figure 1.4. One `Simulation.tick`, 0.1 s of city, left to right. What the observers report is all the other layers will ever learn. Fires, floods and witnesses draw from their own random generators, so arming a hazard does not change the traffic of a seed.

1.2.3 Simulation model

Each engine is a small module with one job, wired together in `sim.py`. We chose published models where they exist, and the rules are checked by tools that measure the geometry of a run.

Part	Model	What it gives the city, with its main numbers
Cars	Intelligent Driver Model; MOBIL lane changes; kinematic bicycle steered by pure pursuit (Stanley when parking)	50 km/h, 1.5 s headway; cars slow for turns, stop at red and yellow lines, park in real bays and pull out only into a gap
Junctions	fixed-time signals, two car phases then a pedestrian phase (44 s); a junction box per conflicting movement	a car enters only when its exit is clear and nobody is in its path; no two bodies overlap in normal traffic
Pedestrians	social-force goal term; time-to-collision avoidance (Karamouzas et al. 2014); a walk network of sidewalks and crosswalks	1.4 m/s walkers, 3 m/s runners; they cross only on crosswalks, on `walk` and with a gap of 4 s or 10 m
Accidents	body overlap (separating axis) between cars, a car rectangle against a person's disc	reckless road users from two risks: red-light runners crash, non-yielding drivers and careless walkers cause pedestrian hits
Fires	a fire grows from 0 to 1 in 30 s; after 40 s it tries to spread to touching buildings every 15 s (50%)	smoke sensor trips at 3 s; 20 to 400 °C and 420 to 5000 ppm CO_2; drivers and walkers keep 25 m away when they can
Floods	inflow by the rational method Q = C·i·A; a parabolic dip, V = 4/3·W·L·$d^{1.5}$/√H; drain 0.3 m³/s, pump 1.2 m³/s	at 0.20 m a street is closed to cars; walkers turn back at 0.10 m on the sidewalk; at 0.20 m a person is trapped
Units	siren driving: ignore signals (7 m/s through a red), 19 m/s top, claim a junction 45 m out, pass queues on the centre line	out of the bay, to the scene, work it (treat, load, close the road, put out, rescue), hospital, back to the bay
Witnesses	line of sight within 40 m; a reaction delay of 1 to 8 s; a call centre that merges calls within 50 m and 300 s	the location 3 m off (10% of callers 15 to 30 m off), 10% name the wrong category; false reports can be switched on

1.2.4 How a scenario is played

A scenario is random traffic plus a short script that stages one event, fired by what the actors are doing (a light about to change, the water 0.27 m deep) rather than at a fixed time, so it still happens with another seed. After that nothing is scripted: witnesses, calls, verdicts and the drive are the code of the free city. Figure 1.5 follows T1; the tags are what each step leaves on the timeline.

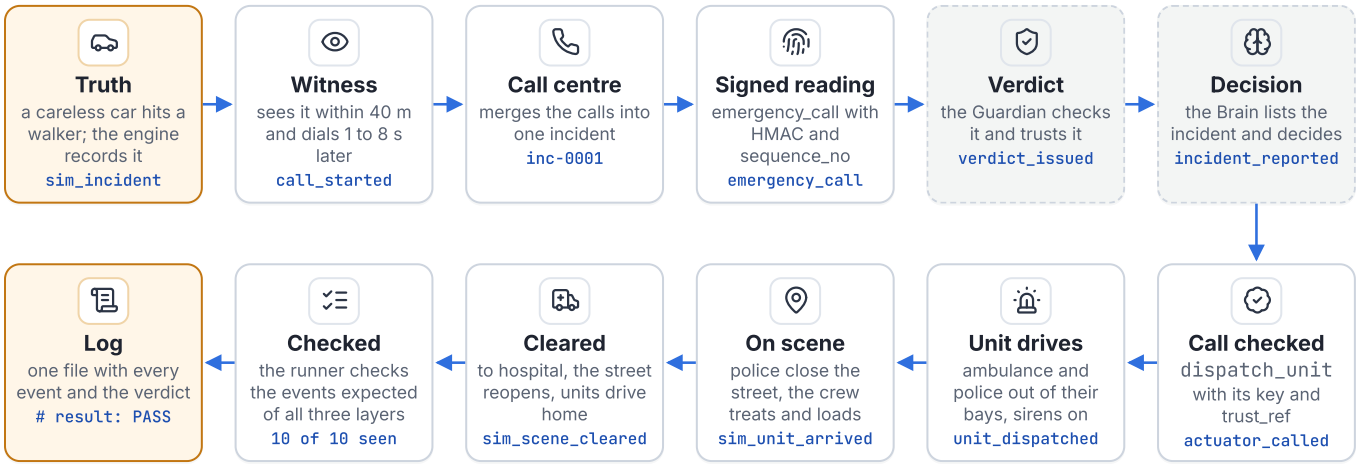


Figure 1.5. One scenario from the truth to its log (T1, about 245 s of simulated time, about a minute at 4×). Dashed cards are other layers. The Brain hears of the accident only when someone sees it and calls.

Every run is one button in the viewer or one command (`docker compose run --rm scenario t1`). The runner switches the mock Brain to decide by itself, or leaves it to a person at its console, waits for the expected events and writes the merged timeline of the three layers to `logs/`. Run for this report on commit `2dc94ec` with the mock Brain deciding, all four pass: T1 saw its 10 expected events and closed at 245 s, T2 10 of 10 at 287 s, T3 9 of 9 at 146 s, T4 12 of 12 at 340 s.



1. Live run or recording, its scenario and clock.
2. Scenarios, city settings, cameras and the People drawer.
3. Counts of cars, people and calls; a card per incident; the three units.
4. In the city: the accident, the call about it, each place by name.
5. Restart, pause, speed from 0.5× to 8×, and the time bar.

Figure 1.6. The viewer in a Busy city run at 8×. Cards say "Nobody is coming yet": no layer has dispatched a unit, so all three are still at their stations. It is the operator's window, drawn only from what the Twin streams.

1.2.5 Decision model: where people go next

As in Phase 0, the model only picks a destination and the engines do the rest. The student is a small model that copies what an expensive LLM (the teacher) would choose, so a decision makes no API call. We generated 6,000 synthetic

decisions: a person (7 traits, a role, driving or walking), the hour and place type, 12 candidate places (category, distance, open or closed, crowd, home or duty place) and 0 to 2 events (fire, crash, closed street). `gpt-4o-mini` answered each with one letter, and its token log-probabilities give a real probability over the 12 options, richer than a single pick (4 minutes, $0.30, no parse failures). Each (person, situation, candidate) triple becomes a row of 58 numbers, the facts in the teacher's prompt. XGBoost learned from 4,800 scenarios (58k rows) the probability that the teacher picks a candidate; 1,200 were kept for testing.

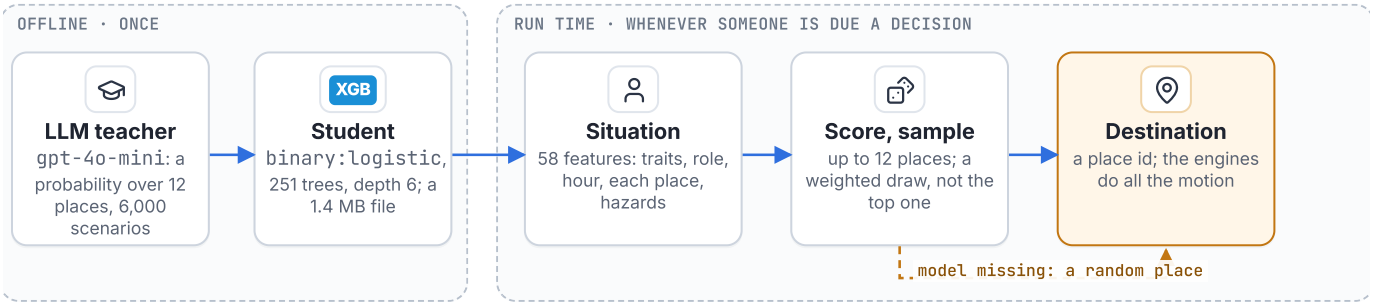


Figure 1.7. The sim scores the candidates and samples one in proportion, so people vary. A simulated day passes in 24 minutes from 08:00.

Match with the teacher, 1,200 held-out scenarios

	Top-1	Top-3
Student	**60.5%**	**90.5%**
Nearest open place	42.7%	not measured
Random	8.3%	25.0%

Share of probability on places to avoid

	Student	Teacher	Chance
Closed places	**10%**	6%	45%
Near an event	**9.1%**	8.1%	14.2%

`test_decision.py`: avoids closed places, fires, crashes, long trips.

1.3 Tech stack and why

At run time everything is on one laptop, with no cloud service. Each logo belongs to its owner.

	Job and technology	Why
	Language and contracts Python 3.12, uv, Pydantic	One language for the engine and the service; a locked build (`uv.lock`); the actuator envelope is a typed model that rejects a bad call with a clear message.
	Service FastAPI, Uvicorn, server-sent events	Typed endpoints with OpenAPI docs at `/docs`; JSON Schema for every actuator in `GET /actuators`; streams without a message broker.
	Simulation NetworkX, NumPy, OSMnx (optional)	Road, turn and walk graphs and their routes; vector geometry. The same engine loads Avenue Habib Bourguiba from OpenStreetMap.
	Decision model XGBoost (CPU build)	Scores every candidate place in one call; the plain wheel would add about 350 MB of GPU libraries, the CPU build does not.
	Viewer React 19, three.js, React Three Fiber, Vite	The 3D city and the timeline draw only what the Twin streams; the same code plays a recording made by the engine.
	Operations Docker Compose, nginx, pytest, Vitest	One command starts the Twin, the viewer and both mocks; nginx adds the operator key so it never reaches a browser; 303 Python and 94 Vitest tests.

1.4 Interfaces in/out

The Interface Card is `twin/INTERFACE.md`. Messages follow the Guardian team's draft dictionary (`2026-09-17`); fields we added are marked *proposed* there.

Sensors: what the Twin sends (rain and water per street)

Type	Sensors	When
`emergency_call`	`ecall-zone-03` …	on each call
`traffic_camera`	`cam-n2`, `cam-n5`	every second
`cctv_anomaly`	`cam-n2-cctv` …	on an anomaly
`building_fire`	`fire-bld-01-03` …	while it smokes
`air_quality`	`aq-zone-01` …	every second
`rainfall`	`rain-dip-2-3` …, 7	every second
`water_level`	`wl-dip-2-3` …, 7	every second

Actuators: what the Twin accepts

Action	From	Effect in the city
`dispatch_unit`	Brain	sends a unit to an `incident_id`
`set_barrier_state`	Brain	close or open a street; cars re-route
`set_pump_state`	Brain	a street's pump on or off (1.2 m³/s)
`quarantine_device`, `isolate_sensor`, `clear_quarantine`	Guardian	its readings stop until cleared; the device keeps working

What the jury looks for. Logic apart from partner I/O: the `CitySource` protocol in `city/base.py`, the only thing the API talks to. Settings in configuration: `config.py` reads the environment, and an empty address disables a link. Clear

errors: `400 401 403 409 422` with a sentence (`test_api.py`, `test_actuator_catalog.py`). Runs alone with mocks (R1): `docker compose up` (`test_interop_guardian_mock.py`, `test_pipeline_e2e.py`); with no partner at all it still simulates.

Direction	Transport and contract	Behaviour a partner can rely on
Out, to the Guardian	`POST {GUARDIAN_INGEST_URL}`, one signed reading per request, every 1 s; also `GET /stream/readings` (SSE)	sequence numbers rise per sensor; a partner that is down costs a `delivery_failed` event, never a stalled city
Out, to the Brain	`POST {BRAIN_NOTIFY_URL}` `readings_available` after each batch	no data in the notice: the Brain reads only what the Guardian has judged
In, from either	`POST /actuators/call` with `X-Api-Key`, `requesting_layer`, `justification`, `trust_ref`; optional `idempotency_key`	answers `dispatched`, `duplicate` or `rejected` with a reason; errors `400 401 403 409 422` with a sentence in `detail`

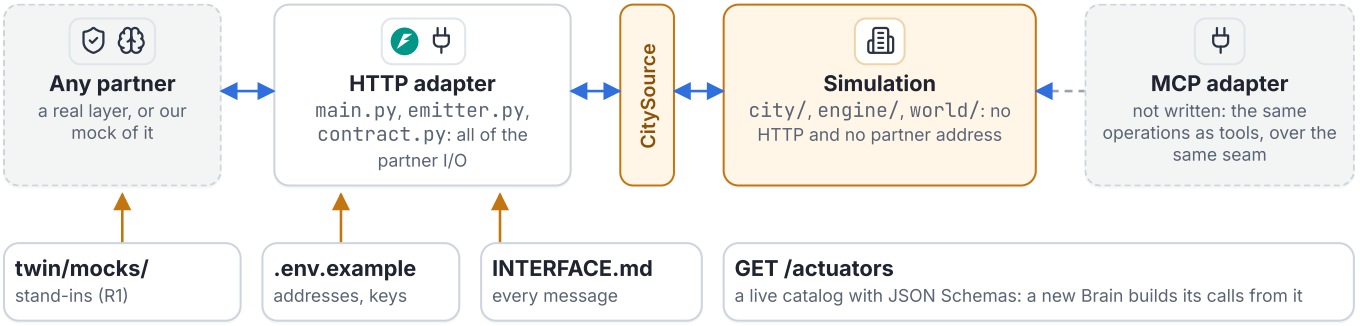


Figure 1.8. Where the logic ends and partners begin. Phase 2 reshuffles the triplets, so a new Guardian or Brain must need only a new address, or at most a new adapter, and never a change to the simulation.

1.5 Security measures

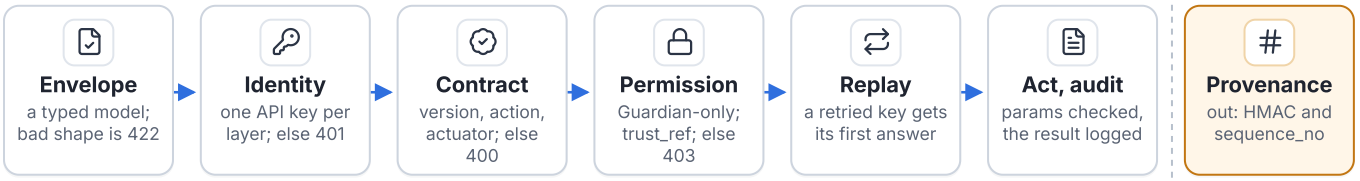


Figure 1.9. Six checks on every incoming call, in the order the code runs them; a call stops at the first that fails, with a sentence saying why. Right of the line: what protects outgoing readings.

Boundary	Control, and the test that keeps it true
Twin to Guardian: readings	HMAC-SHA256 over canonical JSON (sorted keys, no spaces, UTF-8) with `TWIN_HMAC_KEY`; sequence numbers strictly rising per sensor, so drops, replays and edits show (`test_signing.py`, `test_interop_guardian_mock.py`)
Brain and Guardian to Twin: calls	a key per layer, so `requesting_layer` is proven and not just claimed; a Brain call without `trust_ref` is refused (403) and containment is Guardian-only (`test_api.py`); refusals of a key or a permission are logged as `actuator_rejected`
Ground truth	what really happened goes only to the operator's `/timeline`; partners' `/events` leave it out, so a layer cannot cheat its way to a verdict
Operator, secrets	scenarios, simulators and live control need `X-Operator-Key`, which nginx adds so it never reaches a browser; all keys come from the environment, never from code

1.6 Known limitations

- **No joint run yet.** Every scenario passed against our mocks only. The Brain team plans MCP by default with HTTP as an alternative, and the Guardian's `/v1/readings`; addresses are configuration, but paths and event names still need aligning.
- **No falsified readings yet.** The truth is kept apart, but a fake reading cannot yet be sent on one trigger, as bad-data scenarios need; the Test Book has no layer-failure scenario either.
- **Trust stops at presence.** The Twin checks that `trust_ref` is there, not that the verdict exists; that needs a way to ask the Guardian.
- **No TLS.** Shared keys with no rotation and dev defaults in Compose; read endpoints are open, and so are the viewer's controls.
- **Part of the dictionary.** No power-load sensor, and no `set_signal_phase`, `set_valve_state` or `set_grid_connection`.
- **A small city.** Traffic rules are measured on two blocks only. Junctions trade about 23% of car trips for safety, and nobody drives round a wreck.
- **HTTP, not MCP.** The MCP adapter is not written. The student has no LLM fallback or drift check at run time, and its training pipeline is not in this repository.